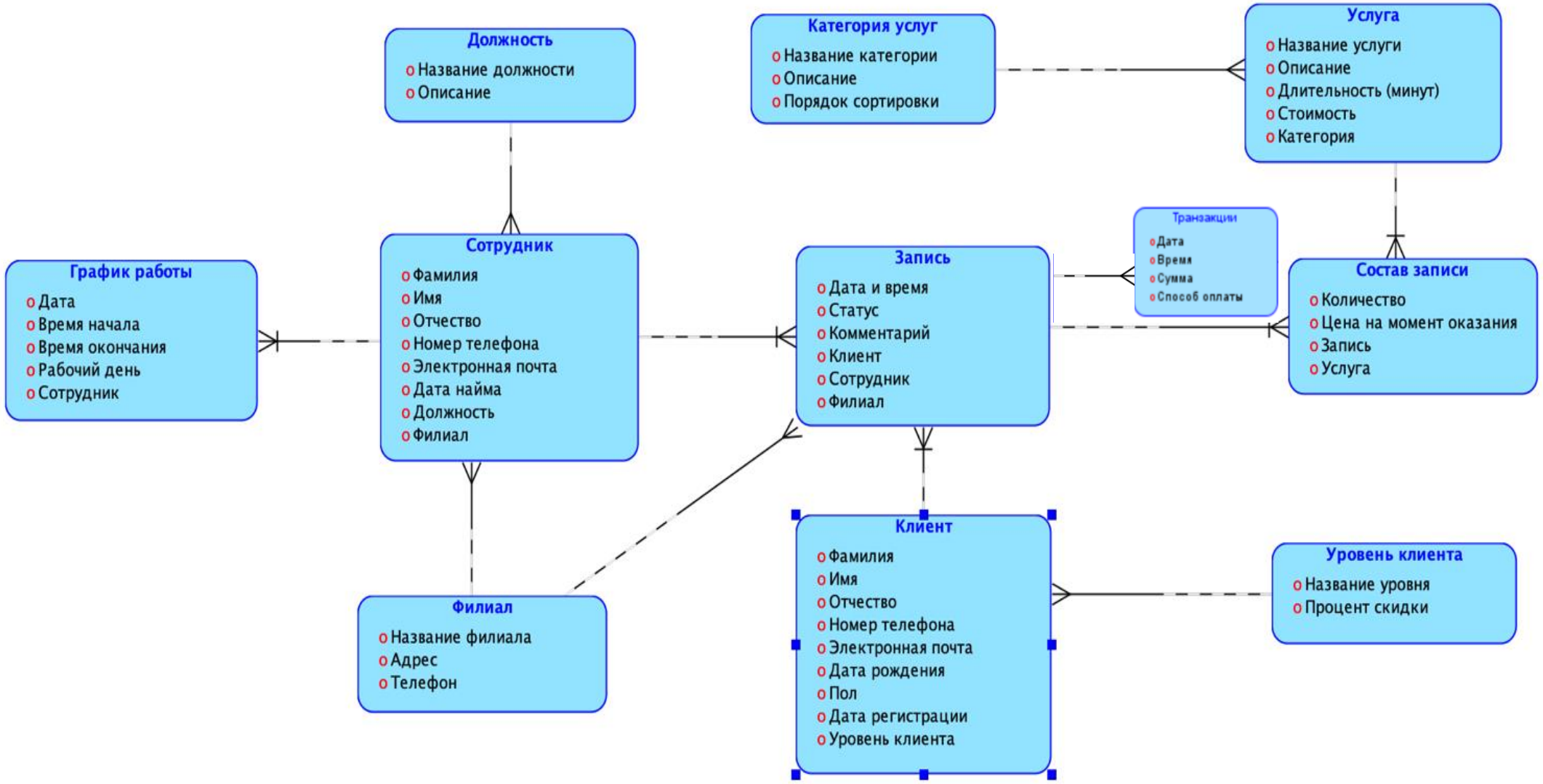
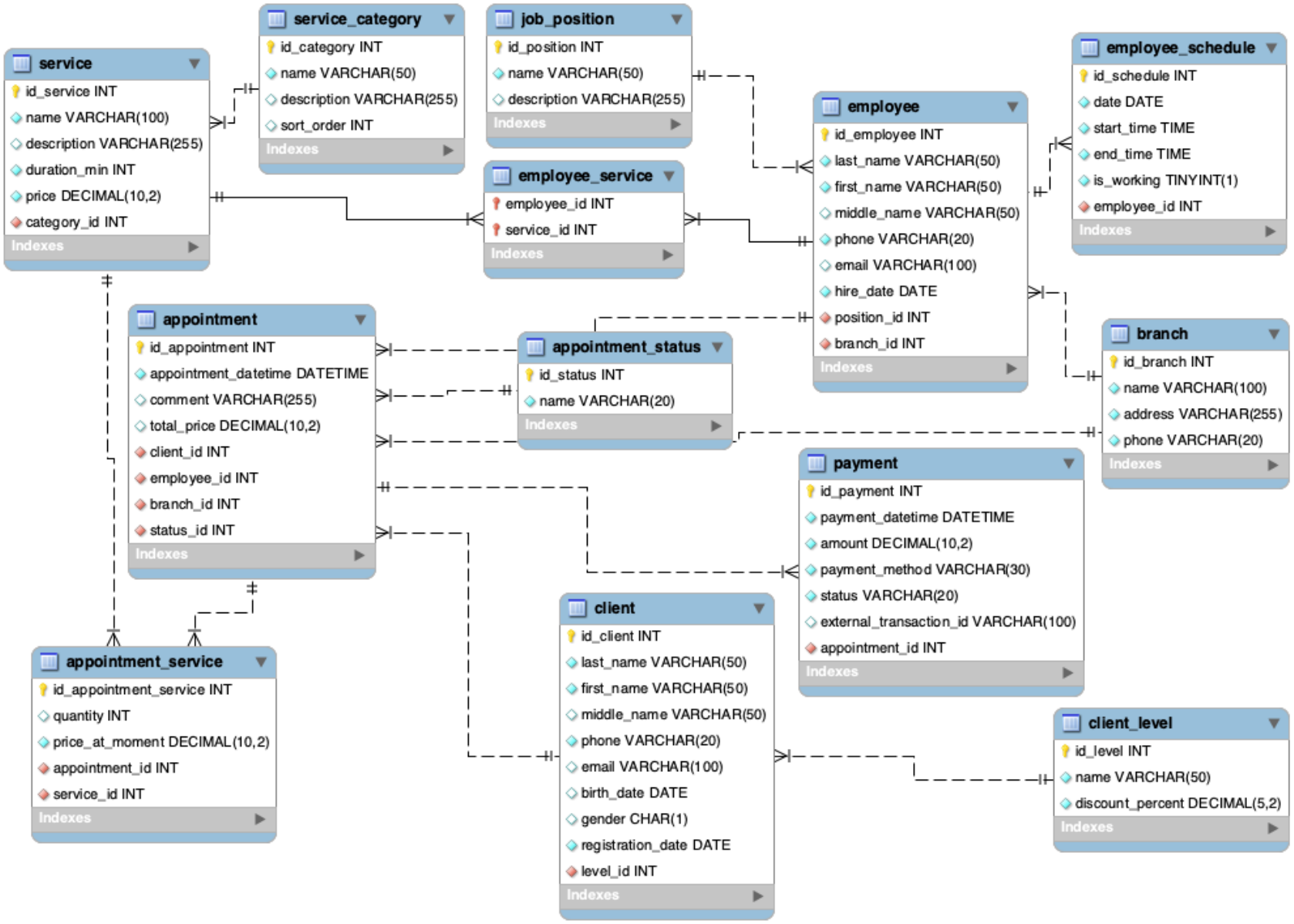
Основные сущности базы данных

Уровень лояльности: Название уровня, Процент скидки
Категория услуги: Название категории, Описание, Порядок сортировки
Должность: Название должности, Описание обязанностей
Статус записи: Название статуса
Филиал: Название филиала, Адрес, Контактный телефон
Сотрудник: Фамилия, Имя, Отчество, Телефон, Электронная почта, Дата найма, Должность, Филиал
Клиент: Фамилия, Имя, Отчество, Телефон, Электронная почта, Дата рождения, Пол, Дата регистрации, Уровень лояльности
Услуга: Название услуги, Описание, Длительность (мин), Стоимость, Категория услуги
Услуга сотрудника: Сотрудник, Услуга
График работы: Дата, Время начала, Время окончания, Признак рабочего дня, Сотрудник
Запись на прием: Дата и время визита, Комментарий, Итоговая стоимость, Клиент, Сотрудник, Филиал, Статус записи
Состав записи: Количество услуг, Цена на момент оказания, Запись, Услуга
Платеж: Дата и время оплаты, Сумма, Способ оплаты, Статус платежа, Внешний идентификатор, Запись на прием

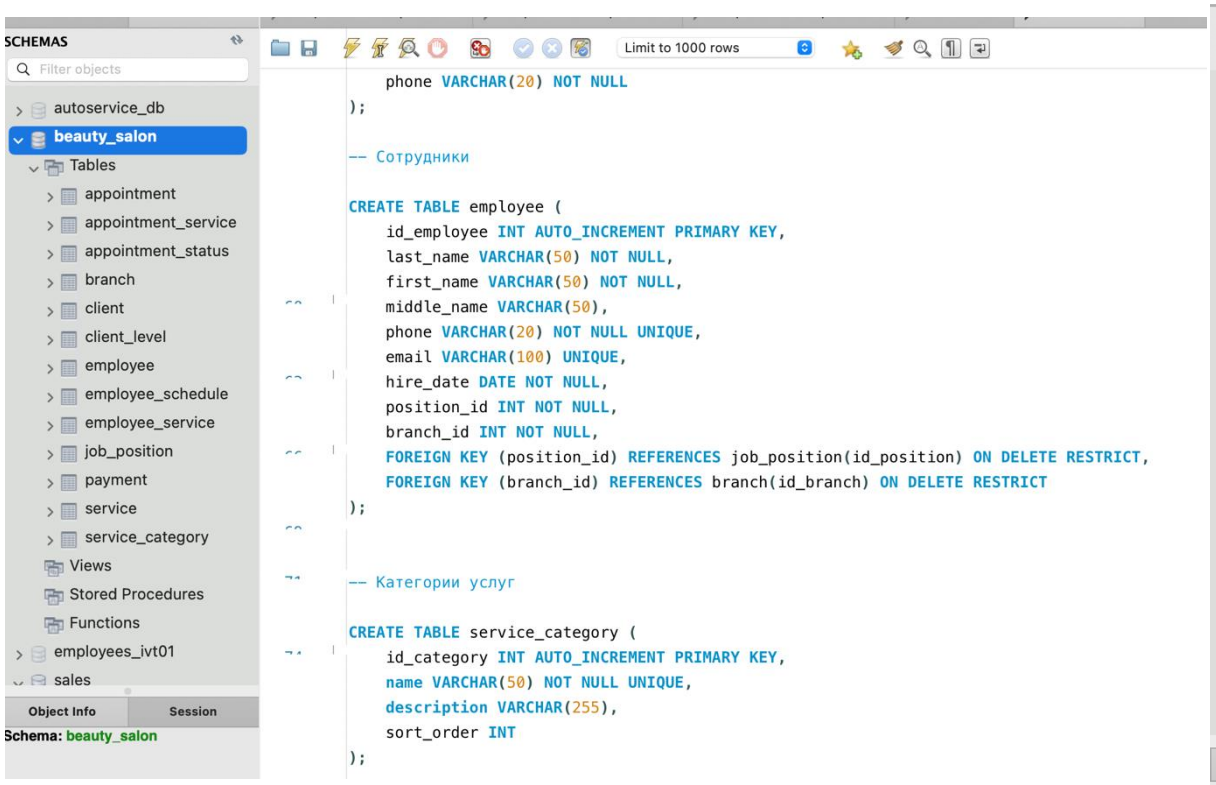
Концептуальная модель

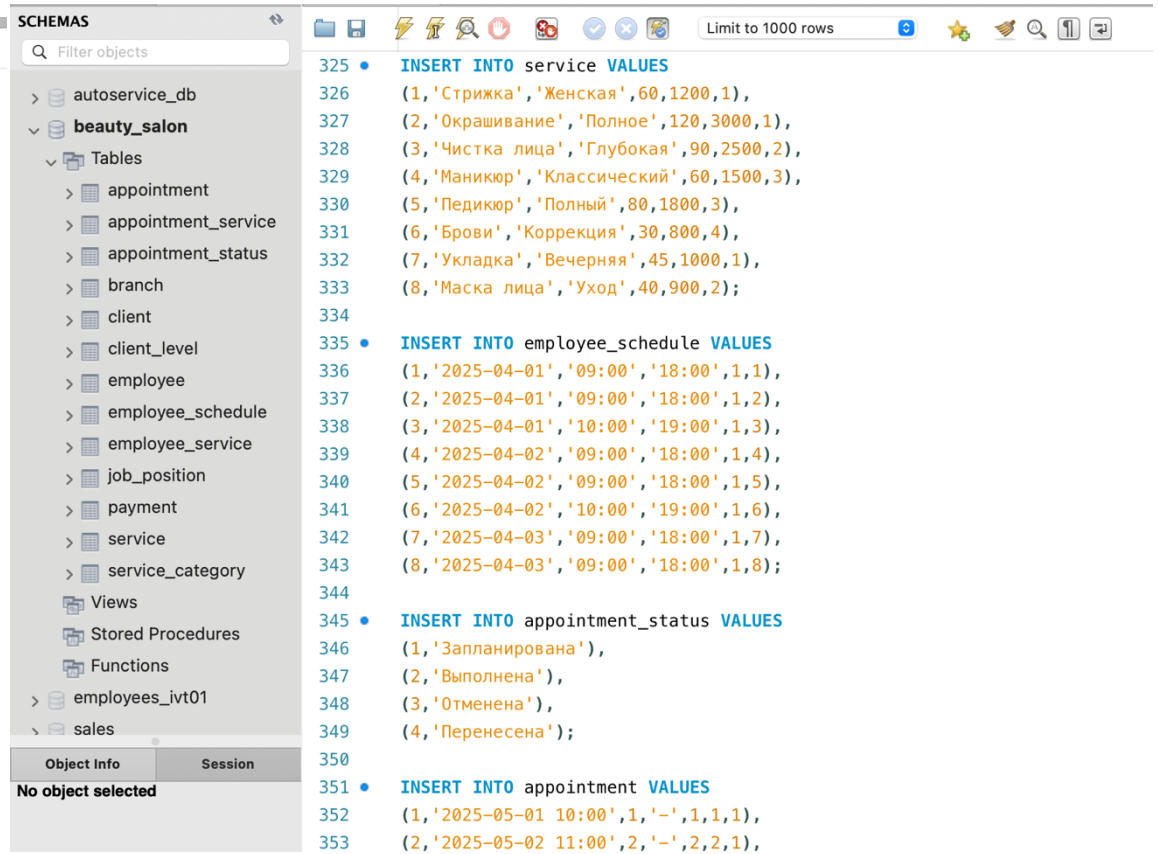


Даталогическая схема



Реализация базы данных через DDL-скрипт





Создание запроса на извлечение данных с использованием CROSS JOIN

```sql
USE beauty_salon;

SELECT
    c.last_name,
    c.first_name,
    a.appointment_datetime,
    e.last_name AS employee_last_name,
    e.first_name AS employee_first_name,
    b.name AS branch_name
FROM client c
CROSS JOIN appointment a
CROSS JOIN employee e
CROSS JOIN branch b
WHERE c.id_client = a.client_id
  AND e.id_employee = a.employee_id
  AND b.id_branch = a.branch_id;
```

100% 20:13

Result Grid Filter Rows: Search Export:

last_name	first_name	appointment_dateti...	employee_last_na...	employee_first_na...	branch_name
Иванова	Анна	2025-05-01 10:00:00	Сидорова	Ольга	Центр
Петров	Илья	2025-05-02 11:00:00	Иванов	Дмитрий	Центр
Морозова	Ольга	2025-05-07 16:00:00	Васильева	Анна	Центр
Иванова	Анна	2025-06-03 09:00:00	Сидорова	Ольга	Центр
Иванова	Анна	2025-06-03 10:00:00	Сидорова	Ольга	Центр
Иванова	Анна	2025-06-03 11:00:00	Сидорова	Ольга	Центр
Иванова	Анна	2025-06-03 12:00:00	Сидорова	Ольга	Центр
Иванова	Анна	2025-06-03 13:00:00	Сидорова	Ольга	Центр
Смирнова	Мария	2025-05-03 12:00:00	Козлова	Ирина	Север
Кузнецов	Максим	2025-05-04 13:00:00	Новиков	Алексей	Север
Волков	Артём	2025-05-08 17:00:00	Михайлов	Игорь	Север
Попова	Елена	2025-05-05 14:00:00	Фёдорова	Мария	Юг
Соколов	Дмитрий	2025-05-06 15:00:00	Белов	Павел	Юг

Выводит список записей с ФИО клиента, датой и временем приёма, ФИО мастера и названием филиала.

Механика (как работает)

CROSS JOIN создаёт декартово произведение четырёх
таблиц: client, appointment, employee, branch.
Фильтр WHERE по равенству ключевых полей оставляет только те комбинации, где запись реально связана с клиентом, сотрудником и филиалом.

Зачем и когда применяется

В рабочих базах данных этот приём не используют – промежуточное декартово произведение огромно. Применяется исключительно в учебных примерах, чтобы показать, что INNER JOIN эквивалентен отфильтрованному CROSS JOIN, и дать понимание внутренней логики соединений.

Создание запроса на извлечение данных с использованием INNER JOIN

Выводит список записей с ФИО клиента, датой и временем приёма, ФИО мастера и названием филиала, но с использованием современного синтаксиса JOIN.

```
395    #SELECT 2.3.2
396 •  SELECT
397        c.last_name,
398        c.first_name,
399        a.appointment_datetime,
400        e.last_name AS employee_last_name,
401        e.first_name AS employee_first_name,
402        b.name AS branch_name
403    FROM client c
404    INNER JOIN appointment a ON c.id_client = a.client_id
405    INNER JOIN employee e ON e.id_employee = a.employee_id
406    INNER JOIN branch b ON b.id_branch = a.branch_id;
407
```

100% 52:406

Result Grid | Filter Rows: Search | Export:

last_name	first_name	appointment_dateti...	employee_last_na...	employee_first_na...	branch_name
Иванова	Анна	2025-05-01 10:00:00	Сидорова	Ольга	Центр
Петров	Илья	2025-05-02 11:00:00	Иванов	Дмитрий	Центр
Морозова	Ольга	2025-05-07 16:00:00	Васильева	Анна	Центр
Смирнова	Мария	2025-05-03 12:00:00	Козлова	Ирина	Север
Кузнецов	Максим	2025-05-04 13:00:00	Новиков	Алексей	Север
Волков	Артём	2025-05-08 17:00:00	Михайлов	Игорь	Север
Попова	Елена	2025-05-05 14:00:00	Фёдорова	Мария	Юг
Соколов	Дмитрий	2025-05-06 15:00:00	Белов	Павел	Юг

Оператор INNER JOIN последовательно соединяет таблицы, оставляя только те записи, для которых находятся соответствия во всех участвующих таблицах. Условие соединения задаётся в секции ON, что отделяет логику связей от фильтров WHERE.

Зачем и когда применяется:
Рекомендованный стандарт SQL-92 для явного указания типа соединения.
Повышает читаемость и упрощает поддержку сложных запросов с большим количеством таблиц.

Создание запроса с использованием процедурных возможностей SQL (команда CASE)

Выводит ФИО клиента, процент его скидки и присваивает текстовый статус лояльности на основе размера скидки.

```sql
#SELECT 2.3.3
SELECT
    c.last_name,
    c.first_name,
    cl.discount_percent,
    CASE
        WHEN cl.discount_percent >= 15 THEN 'VIP'
        WHEN cl.discount_percent >= 10 THEN 'Постоянный'
        WHEN cl.discount_percent >= 5 THEN 'Базовый'
        ELSE 'Новичок'
    END AS client_status
FROM client c
INNER JOIN client_level cl ON c.level_id = cl.id_level;
```

100% 56:420

Result Grid Filter Rows: Search Export:

last_name	first_name	discount_perc...	client_status
Смирнова	Мария	0.00	Новичок
Морозова	Ольга	0.00	Новичок
Иванова	Анна	5.00	Базовый
Попова	Елена	5.00	Базовый
Петров	Илья	10.00	Постоянный
Соколов	Дмитрий	10.00	Постоянный
Кузнецов	Максим	15.00	VIP
Волков	Артём	15.00	VIP

Конструкция CASE работает как условный оператор: последовательно проверяет условия WHEN и возвращает значение после первого истинного THEN. Если ни одно условие не выполнено, срабатывает ELSE.

Применяется для создания вычисляемых категорий на основе числовых или других критериев.
Позволяет в одном запросе получить готовую сегментацию клиентов для отчётов или маркетинговых рассылок.

Запрос с использованием группировок, группировочных функций и условий на группы (HAVING)

Подсчитывает количество записей в каждом филиале и оставляет только те филиалы, где количество записей ≥ 2.

```
422    #SELECT 2.3.4
423 •  SELECT
424        b.name AS branch_name,
425        COUNT(a.id_appointment) AS appointment_count
426    FROM branch b
427    INNER JOIN appointment a ON b.id_branch = a.branch_id
428    GROUP BY b.name
429    HAVING COUNT(a.id_appointment) >= 2;
430
```

100% 67:415

Result Grid | Filter Rows: Search | Export:

branch_name	appointment_co...
Центр	3
Север	3
Юг	2

GROUP BY b.name объединяет строки по названию филиала.
Агрегатная функция COUNT() считает количество записей в каждой группе.
HAVING фильтрует сгруппированные данные после применения агрегатных функций.

Применяется для анализа загруженности филиалов и выявления наиболее активных точек сети.
Когда требуется наложить условие на результат агрегатной функции, что невозможно сделать через WHERE.

Запрос с использованием левого соединения (LEFT JOIN)

Выводит всех клиентов и (если есть) идентификатор и дату их записей. Клиенты без записей отображаются с NULL в полях таблицы appointment.

```
431     #SELECT 2.3.5
432 •   SELECT
433         c.last_name,
434         c.first_name,
435         a.id_appointment,
436         a.appointment_datetime
437     FROM client c
438     LEFT JOIN appointment a ON c.id_client = a.client_id
439     ORDER BY c.last_name, c.first_name;
440
```

100% 49:425

Result Grid Filter Rows: Search Export:

last_name	first_name	id_appointme...	appointment_dateti...
Волков	Артём	8	2025-05-08 17:00:00
Иванова	Анна	1	2025-05-01 10:00:00
Кузнецов	Максим	4	2025-05-04 13:00:00
Морозова	Ольга	7	2025-05-07 16:00:00
Петров	Илья	2	2025-05-02 11:00:00
Попова	Елена	5	2025-05-05 14:00:00
Смирнова	Мария	3	2025-05-03 12:00:00
Соколов	Дмитрий	6	2025-05-06 15:00:00

LEFT JOIN берёт все строки из левой таблицы (client) и пытается найти соответствие в правой (appointment) по условию ON. Если соответствия нет, поля правой таблицы заполняются значениями NULL.

Зачем и когда применяется:
Для выявления клиентов, ещё не воспользовавшихся услугами (аудитория для привлечения).
Когда необходимо сохранить все записи ведущей таблицы независимо от наличия связанных данных.

Запрос с использованием вложенного подзапроса (вложенный SELECT)

Выводит ФИО клиентов, имеющих хотя бы одну запись на приём после указанной даты (в примере – после 1 мая 2025 года).

```
41    #SELECT 2.3.6
42    SELECT
43        c.last_name,
44        c.first_name
45    FROM client c
46    WHERE c.id_client IN (
47        SELECT a.client_id
48        FROM appointment a
49        WHERE a.appointment_datetime > '2025-05-01 00:00:00'
50    );
51
```

0% 27:436

Result Grid Filter Rows: Search Export:

last_name	first_name
Иванова	Анна
Петров	Илья
Смирнова	Мария
Кузнецов	Максим
Попова	Елена

Внутренний подзапрос (в скобках) выполняется первым и возвращает список client_id клиентов, удовлетворяющих условию по дате. Внешний запрос фильтрует таблицу client по этому списку через оператор IN.

Зачем и когда применяется:
Когда критерий отбора зависит от агрегированных или вычисляемых данных другой таблицы.
Позволяет избежать дублирования строк при использовании JOIN, если нужны только уникальные сущности.

Запрос на создание представления (VIEW)

Создаёт виртуальную таблицу view_client_status, которая автоматически возвращает ФИО клиента, его скидку и текстовый статус лояльности.

```
452     #SELECT 2.3.7
453 •   CREATE VIEW view_client_status AS
454     SELECT
455         c.first_name,
456         c.last_name,
457         cl.discount_percent,
458         CASE
459             WHEN cl.discount_percent >= 15 THEN 'VIP'
460             WHEN cl.discount_percent >= 10 THEN 'Постоянный'
461             WHEN cl.discount_percent >= 5 THEN 'Базовый'
462             ELSE 'Новичок'
463         END AS client_status
464     FROM client c
465     INNER JOIN client_level cl ON c.level_id = cl.id_level;
```

100% 3:450

Result Grid | Filter Rows: Search | Export:

first_name	last_name	discount_perc...	client_status
Мария	Смирнова	0.00	Новичок
Ольга	Морозова	0.00	Новичок
Анна	Иванова	5.00	Базовый
Елена	Попова	5.00	Базовый
Илья	Петров	10.00	Постоянный
Дмитрий	Соколов	10.00	Постоянный
Максим	Кузнецов	15.00	VIP
Артём	Волков	15.00	VIP

Представление (VIEW) сохраняет только определение запроса, а не физические данные. При обращении к представлению СУБД каждый раз выполняет сохранённый SELECT, возвращая актуальные данные на текущий момент.

Зачем и когда применяется:
Для упрощения часто используемых сложных запросов.
Для сокрытия внутренней структуры базы данных от конечных пользователей или приложений.
Для создания унифицированного источника данных для отчётов и аналитики.

Создание триггера limit_appointments_per_employee

Триггер на ограничение количества записей на сотрудника в день

```
859 •  CREATE TRIGGER limit_appointments_per_employee
860    BEFORE INSERT ON appointment
861    FOR EACH ROW
862    BEGIN
863        DECLARE appointment_count INT;
864
865        SELECT COUNT(*)
866        INTO appointment_count
867        FROM appointment
868        WHERE employee_id = NEW.employee_id
869          AND DATE(appointment_datetime) = DATE(NEW.appointment_datetime);
870
871        IF appointment_count >= 5 THEN
872            SIGNAL SQLSTATE '45000'
873            SET MESSAGE_TEXT = 'Превышено максимальное количество записей (5) на сотрудника в день';
874        END IF;
875    END //
876
877    DELIMITER ;
878
879 •  SHOW TRIGGERS LIKE 'appointment';
```

Перед вставкой новой записи в таблицу appointment проверяет, сколько активных записей уже есть у данного сотрудника на указанную дату. Если количество достигает или превышает 5, операция блокируется, и СУБД возвращает ошибку. В противном случае вставка разрешается.

Триггер срабатывает перед (BEFORE INSERT) добавлением каждой новой строки. Он считывает значения NEW.employee_id и NEW.appointment_datetime из ещё не вставленной записи, выполняет SELECT COUNT(*), подсчитывая уже существующие записи этого сотрудника на эту же дату (исключая отменённые). Если полученное число cnt >= 5, вызывается команда SIGNAL, которая генерирует ошибку и откатывает операцию вставки. Если лимит не превышен, вставка выполняется штатно.

Применяется для защиты от перегрузки мастеров и обеспечения равномерной загрузки персонала. Для автоматического соблюдения бизнес-правила «не более 5 клиентов в день на одного сотрудника» без необходимости проверок на стороне приложения. При любых попытках вставки (через приложение, API или прямо в БД) правило будет единообразно соблюдаться.

Создание триггера set_registration_date

Триггер на дату регистрации клиента

```
DELIMITER //

CREATE TRIGGER set_registration_date
BEFORE INSERT ON client
FOR EACH ROW
BEGIN
    IF NEW.registration_date IS NULL THEN
        SET NEW.registration_date = CURDATE();
    END IF;
```

Перед вставкой нового клиента в таблицу client проверяет, задано ли значение registration_date. Если поле не заполнено (NULL), триггер автоматически проставляет текущую дату.

Триггер срабатывает перед вставкой строки (BEFORE INSERT). Он анализирует значение NEW.registration_date. Если оно равно NULL, то с помощью конструкции SET NEW.registration_date = CURDATE() подменяет его текущей датой. Если в запросе INSERT дата была передана явно, она остаётся без изменений. Затем вставка продолжается уже с модифицированным или исходным значением.

Зачем и когда применяется:
Для гарантии заполнения обязательного поля (избегаем ошибок нарушения целостности при случайной вставке NULL).
Для автоматизации учёта: регистрационная дата всегда будет корректной, даже если клиентский интерфейс не передал её.
Упрощает логику приложения, снимая с него обязанность всегда вычислять текущую дату.

Создание хранимой процедуры GetClientAppointments

Хранимая процедура получения записей клиента

```
DELIMITER //
CREATE PROCEDURE GetClientAppointments(IN p_client_id INT)
BEGIN
    SELECT
        a.appointment_datetime AS 'Дата и время',
        s.name AS 'Статус',
        CONCAT(e.last_name, ' ', LEFT(e.first_name, 1), '.') AS 'Сотрудник',
        b.name AS 'Филиал',
        COALESCE(SUM(aps.price_at_moment * aps.quantity), 0) AS 'Итоговая стоимость, руб.'
    FROM appointment a
    JOIN appointment_status s ON a.status_id = s.id_status
    JOIN employee e ON a.employee_id = e.id_employee
    JOIN branch b ON a.branch_id = b.id_branch
    LEFT JOIN appointment_service aps ON a.id_appointment = aps.appointment_id
    WHERE a.client_id = p_client_id
    GROUP BY a.id_appointment, a.appointment_datetime, s.name, e.last_name, e.first_name, b.name
    ORDER BY a.appointment_datetime DESC;
END //
DELIMITER ;

CALL GetClientAppointments(1);
```

Возвращает полный список записей на приём для конкретного клиента: дату и время визита, текущий статус записи, фамилию и инициалы мастера, название филиала и итоговую стоимость.

Процедура объявляется с одним входным параметром – идентификатором клиента. В теле процедуры выполняется запрос SELECT с несколькими JOIN, который собирает необходимые данные из
таблиц appointment, appointment_status, employee и branch. Результат формируется динамически, основываясь на переданном p_client_id. При вызове CALL GetClientAppointments(1) СУБД выполняет процедуру и возвращает результирующий набор строк.

Зачем и когда применяется:
Для быстрого получения истории визитов клиента администратором или в интерфейсе личного кабинета.
Инкапсулирует сложную логику соединений, позволяя вызывать готовую процедуру без повторения громоздкого SQL в приложении.
Может использоваться для построения отчётов по активности конкретных клиентов.

Создание хранимой процедуры GetEmployeeScheduleByDate

Хранимая процедура получения графика сотрудника

```
952  #Хранимая процедура получения графика сотрудника.Удаляем процедуру, если она уже существует
953  DROP PROCEDURE IF EXISTS GetEmployeeScheduleByDate;
954  DELIMITER //
955  CREATE PROCEDURE GetEmployeeScheduleByDate(
956      IN p_employee_id INT,
957      IN p_date DATE
958  )
959  BEGIN
960      SELECT
961          date AS 'Дата',
962          start_time AS 'Начало работы',
963          end_time AS 'Окончание работы',
964          CASE WHEN is_working = 1 THEN 'Рабочий день' ELSE 'Выходной' END AS 'Статус'
965      FROM employee_schedule
966      WHERE employee_id = p_employee_id AND date = p_date;
967  END //
968  DELIMITER ;
969  #Вызов процедуры для сотрудника с id = 1 на дату '2025-04-01'
970  CALL GetEmployeeScheduleByDate(1, '2025-04-01');
```

100% 52:953

Result Grid Filter Rows: Search Export:

Дата	Начало работы	Окончание работы	Статус
2025-04-01	09:00:00	18:00:00	Рабо...

Возвращает рабочий график сотрудника на конкретную дату: время начала и окончания работы, а также признак того, является ли день рабочим.

Процедура принимает два параметра: идентификатор сотрудника и дату. В теле выполняется простой SELECT из таблицы employee_schedule с фильтрацией по обоим параметрам. Так как у сотрудника на одну дату может быть только одна запись графика, результатом будет либо одна строка с часами работы, либо пустой набор (если график на эту дату не задан). При вызове CALL GetEmployeeScheduleByDate(5, '2025-05-15') СУБД возвращает соответствующие данные.

Зачем и когда применяется:
Для проверки доступности мастера перед созданием новой записи на приём (чтобы не назначить клиента в нерабочее время).
Для отображения расписания работы сотрудников в интерфейсе администратора.
Автоматизирует рутинный запрос, который может многократно использоваться в разных частях системы.